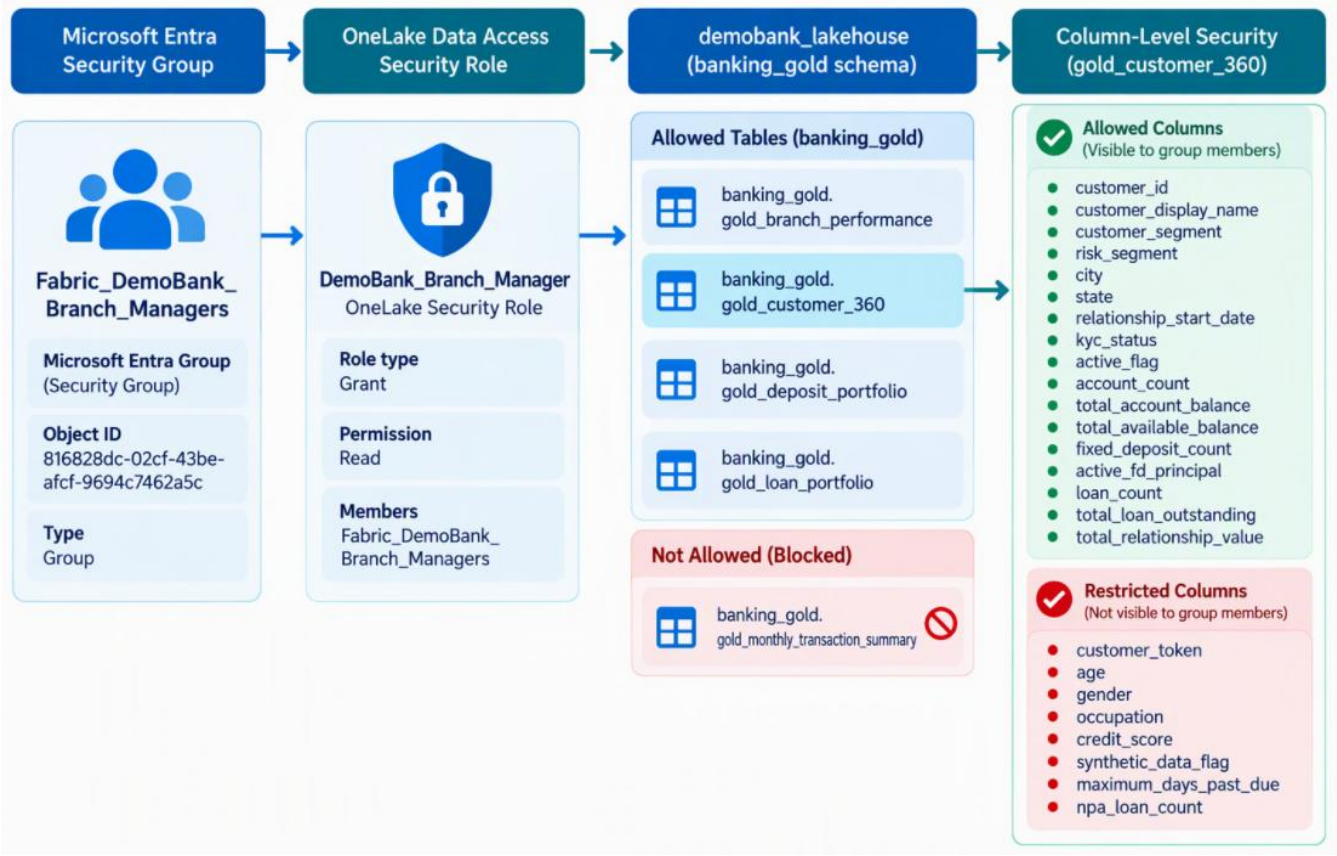
Microsoft Fabric OneLake Data Access Security

Customer-Neutral Implementation Guide

All customer and environment-specific values are represented as XXXX.

Scope	Outcome
Identity	Microsoft Entra group mapped to a OneLake role
Table access	Selected Lakehouse tables are permitted
Column security	A permit-list controls visible columns
Deployment	REST API validation first, then explicit apply

Prepared By: Rakesh Sharma| Solution Engineer | Microsoft



1.

Purpose and outcome

This guide provides a reusable pattern for configuring Microsoft Fabric OneLake Data Access Security. It is intentionally anonymized for publication in a Git repository. Replace each XXXX value only in a private customer-specific branch or deployment pipeline.

Setting	Value
Lakehouse	XXXX
Schema	XXXX
Workspace ID	XXXX
Tenant ID	XXXX
Security group	XXXX
Group object ID	XXXX
OneLake role	XXXX
Lakehouse item ID	XXXX

2. Security design

The design follows least privilege: group membership controls identity, table paths define object access, and a column permit-list narrows access to sensitive tables.

Control	Implementation
Identity	Microsoft Entra security group, XXXX
Table security	Approved table paths use XXXX placeholders
Column security	Approved column names remain explicit; customer-specific

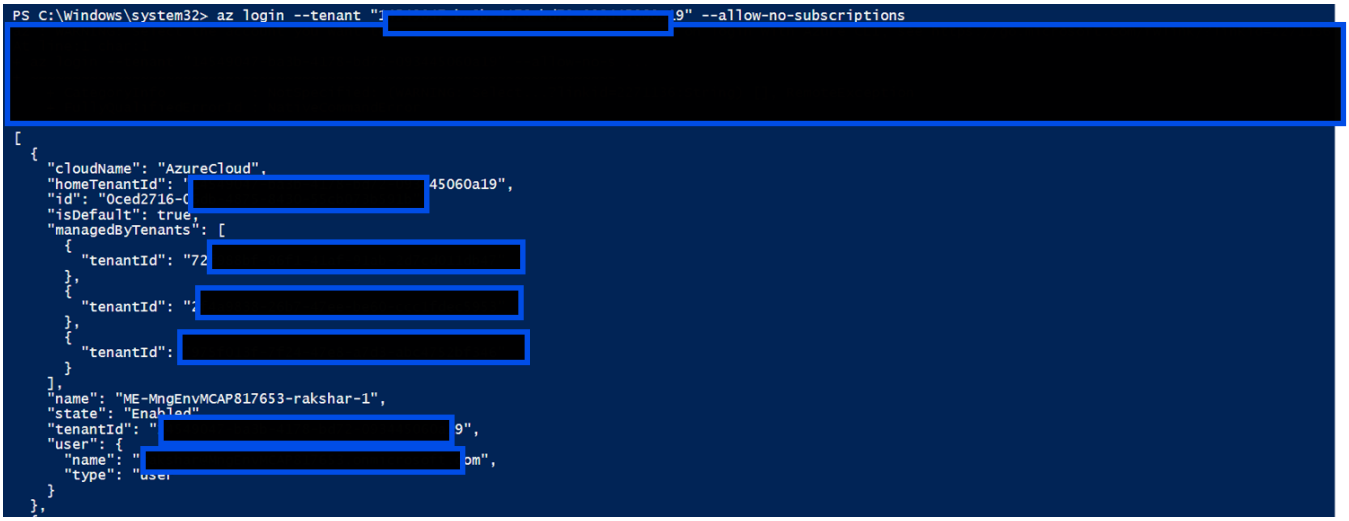
	names use XXXX
Validation	Use dryRun=true before deployment
Testing	Use a non-elevated test user

3. Visual walkthrough

The original screenshots contained tenant IDs, workspace IDs, customer names, local user paths, item names, and bearer-token output. For safe publication, the raw captures were removed and replaced with the following ordered, sanitized command panels.

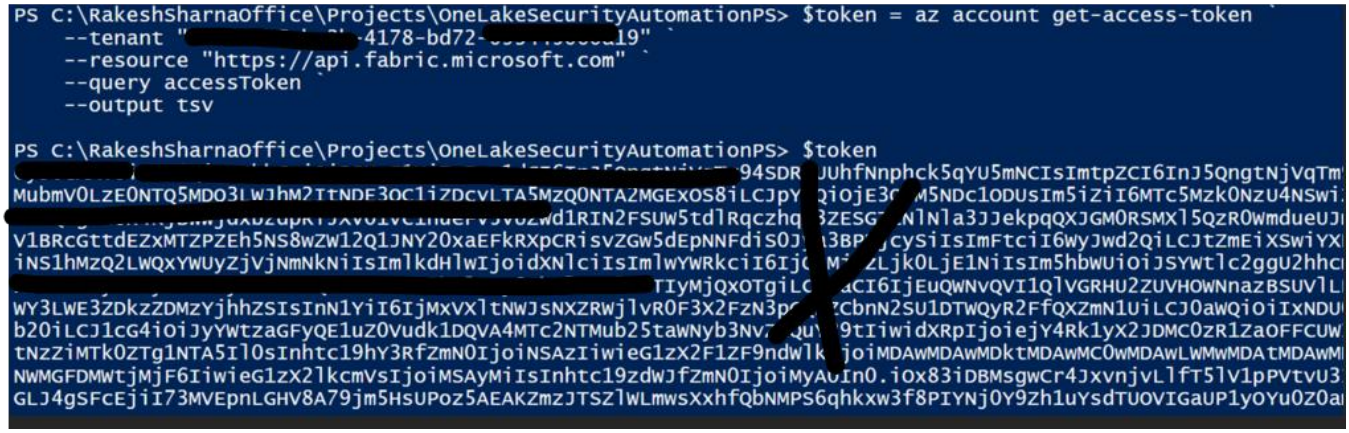
Step 1 - Sign in to the target tenant

```
az login --tenant "XXXX" --allow-no-subscriptions
```



Step 2 - Obtain a Fabric access token

```
$token = az account get-access-token `
  --tenant "XXXX" `
  --resource "https://api.fabric.microsoft.com" `
  --query accessToken `
  --output tsv
# Never print or commit the token. Output: XXXX
```



Step 3 - Verify the workspace

```
$workspaceId = "XXXX"
Invoke-RestMethod -Uri "https://api.fabric.microsoft.com/v1/workspaces/$workspaceId" -Headers $headers
```

```
PS C:\RakeshSharnaOffice\Projects\OneLakeSecurityAutomationPS> $workspaceId = "[redacted]b63b-c85b78442be0"

PS C:\RakeshSharnaOffice\Projects\OneLakeSecurityAutomationPS> $workspace = Invoke-RestMethod `
    -Uri "https://api.fabric.microsoft.com/v1/workspaces/$workspaceId" `
    -Method GET `
    -Headers $headers

PS C:\RakeshSharnaOffice\Projects\OneLakeSecurityAutomationPS> $workspace | Format-List id, displayName, type


id          : [redacted]1-b63b-c85b78442be0
displayName : rsfabricworkspace
type        : Workspace
```

Step 4 - Verify the Lakehouse item

```
$itemId = "XXXX"
Invoke-RestMethod -Uri "https://api.fabric.microsoft.com/v1/workspaces/$workspaceId/items/$itemId" -
Headers $headers
```

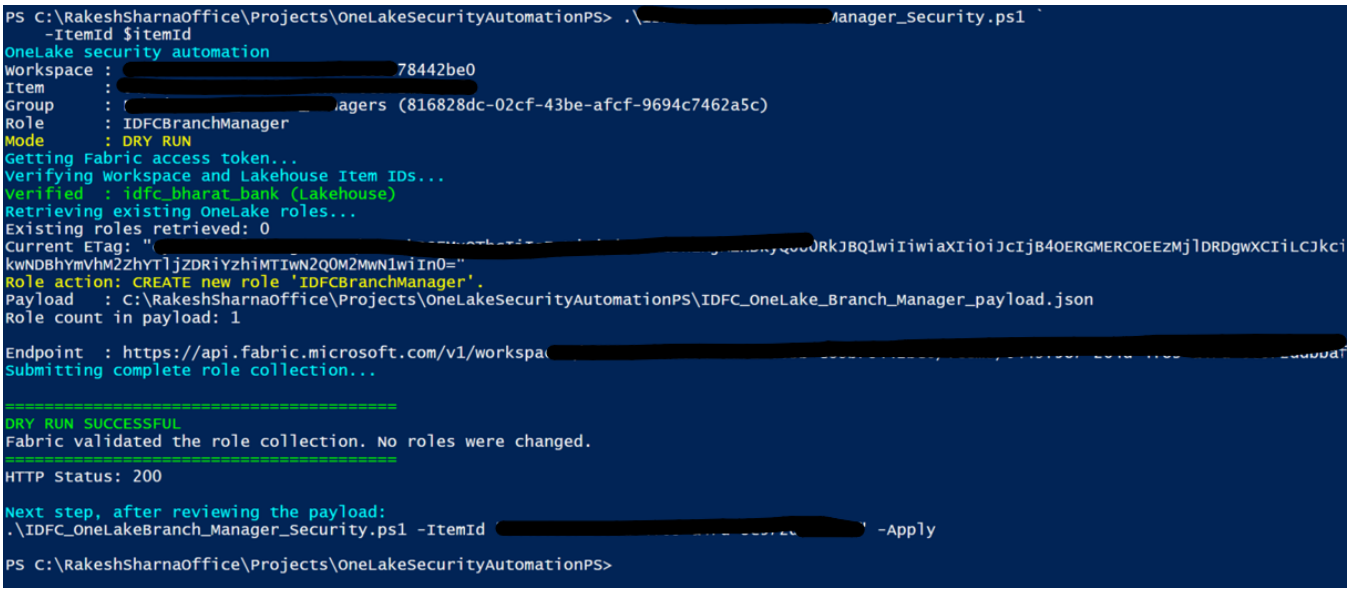
```
PS C:\RakeshSharnaOffice\Projects\OneLakeSecurityAutomationPS> $items = Invoke-RestMethod `
    -Uri "https://api.fabric.microsoft.com/v1/workspaces/$workspaceId/items" `
    -Method GET `
    -Headers $headers

PS C:\RakeshSharnaOffice\Projects\OneLakeSecurityAutomationPS> $items.value |
    Select-Object displayName, type, id |
    Sort-Object type, displayName |
    Format-Table -AutoSize

displayName                                type                 id
-----------                                ----                 --
CopyLH2WH                                  CopyJob              [redacted]3cb92169cc
ABCLAIAssitant                             DataAgent            [redacted]ba3a9fb318
Agent2S3                                   DataAgent            [redacted]08df7144c7
DemoBajaj                                  DataAgent            [redacted]6e989dd9d3
FabAgentBenchMarking                       DataAgent            [redacted]ee703b110b
IDFC-360-BOT                               DataAgent            [redacted]e3c680652b
IDFCAiAgent                                DataAgent            [redacted]4a89d08cc2
Idfc-Leads-Agent                           DataAgent            [redacted]0ec1a9cbec
IDFCSmartAssist                            DataAgent            [redacted]b90391a988
rsFabdataagent                             DataAgent            [redacted]ace6a17141
SalesAgent                                 DataAgent            [redacted]7bca7c8f37
SAMPLEAGENT                                DataAgent            [redacted]795ceea0f8
TradingAssitance                           DataAgent            [redacted]5255c2f4ad
Dataflow 1                                 Dataflow             [redacted]4abd62b839
ReconcilationProcess                       Dataflow             [redacted]c02f6c5c01
ABCLDemoPipeline                           DataPipeline         [redacted]fd27119c99
Connect2DV                                 DataPipeline         [redacted]197290cf3c
GetmyDatafromSQL                           DataPipeline         [redacted]fa65551ae7
Pipeline_1                                 DataPipeline         [redacted]0e19b1dd34
PipelineDemo                               DataPipeline         [redacted]fc86e404f8
Reconcilation                              DataPipeline         [redacted]ef705588a3
RefreshHourlyWarhehouse                    DataPipeline         [redacted]0fcc149f2a
RequirementEnv                             Environment          [redacted]23672d21f3
ABCLMFEH                                   Eventhouse           [redacted]f58bf8d0a0
```

Step 5 - Run validation first

```
.\scripts\Deploy-OneLakeDataAccessRole.ps1 -ItemId "XXXX"
# Expected mode: DRY RUN
```



Step 6 - Apply after review

```
.\scripts\Deploy-OneLakeDataAccessRole.ps1 -ItemId "XXXX" -Apply
```

4. Prerequisites

- PowerShell 7
- Azure CLI
- A Fabric workspace containing the target Lakehouse
- A Microsoft Entra security group
- Permissions to manage OneLake Data Access Security
- A non-elevated test user

```
winget install --id Microsoft.PowerShell --source winget
winget install --exact --id Microsoft.AzureCLI
pwsh --version
az version
```

5. Configure the script

Open scripts/Deploy-OneLakeDataAccessRole.ps1 and replace only the intended XXXX placeholders. Keep secrets and bearer tokens outside source control.

Parameter or value	Repository placeholder
Tenant ID	XXXX
Workspace ID	XXXX
Lakehouse item ID	XXXX
Group object ID	XXXX
Role name	XXXX
Table paths	XXXX
Column permit-list	Explicit metadata values or XXXX

6. Validate and deploy

6.1 Dry run

```
.\scripts\Deploy-OneLakeDataAccessRole.ps1 -ItemId "XXXX"
```

Review the resolved item, current roles, generated payload, and Fabric validation response. The dry run must succeed before applying changes.

6.2 Apply

```
.\scripts\Deploy-OneLakeDataAccessRole.ps1 -ItemId "XXXX" -Apply
```

7. Test access

Test	Expected result
Approved table query	Succeeds
Approved column query	Succeeds
Restricted column query	Fails
Excluded table query	Fails
Elevated workspace user test	Not valid for restricted-reader verification

8. Troubleshooting

Symptom	Likely cause	Action
401 Unauthorized	Token, tenant, or resource mismatch	Sign in again and request a Fabric API token
403 Forbidden	Insufficient permissions	Confirm delegated scope and item permissions
404 ItemNotFound	Wrong workspace or item ID	Recheck the Lakehouse item GUID
400 validation error	Invalid path, casing, role name, or payload	Use dry run and compare exact metadata
409 or 412	ETag changed	Retrieve current roles and retry with the latest ETag
User sees too much	Elevated workspace role or another broad role	Review all effective roles and workspace assignments

9. Git publication checklist

- Search the repository for GUIDs, email addresses, customer names, local user paths, and tokens.
- Confirm all environment-specific values are XXXX.
- Do not commit generated payload JSON or log files.
- Keep dry-run-first behavior enabled.
- Validate the Word guide visually before publishing.

10. Disclaimer

This repository is a sample/reference implementation, not an official supported deployment tool. Validate the configuration in a development or test environment and consult current Microsoft Fabric documentation before production use.